Отладка и тестирование

Введение в ПИ (C++)

Debugging & Testing

Стратегия

1. Воспроизвести минимально
2. Сформулировать гипотезу («ломается на пустом вводе»)
3. Проверить фактами: печать / отладчик / assert
4. Исправить одну причину, перепроверить

Запомните

Случайные правки без гипотезы — антипаттерн.

Чтение ошибок компилятора

- Первая ошибка — якорь
- `expected ';'` — часто строка *выше*
- `no matching function` — типы аргументов
- Шаблоны STL дают длинный текст — ищите *свою* строку кода

Временная диагностика

```
1 std::cerr << "i=" << i << " size=" << v.size() << "\n";
```

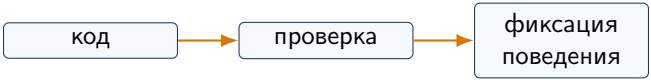
`cerr` — поток ошибок; не смешивается с обычным выводом так же, как лог.

Отладчик *(debugger)* — минимум

- Breakpoint на подозрительной строке
- Step over / step into
- Смотреть локальные переменные

(gdb, lldb, встроенный отладчик IDE — на выбор площадки)

Зачем тесты в ПИ



Тест — исполняемая спецификация маленького куска.

assert

```cpp
#include <cassert>

int abs_int(int x) { return x < 0 ? -x : x; }

int main() {
    assert(abs_int(-3) == 3);
    assert(abs_int(0) == 0);
}
```

В Release `assert` может быть отключён (`NDEBUG`).

Ручной тестовый драйвер

```
int passed = 0, failed = 0;
auto check = [&](bool cond, const char* name) {
    if (cond) { ++passed; }
    else { ++failed; std::cerr << "FAIL " << name << "\n"; }
};
check(clamp(5, 0, 3) == 3, "clamp high");
```

Unit test — идея

- Одна функция / класс — много маленьких случаев
- Границы: 0, пусто, отрицательные, максимумы
- Фреймворки (GoogleTest и др.) — позже; принцип тот же

Викторина 1 — порядок действий

Викторина

Программа падает иногда. Первый разумный шаг?

1. Переписать всё с нуля
2. Найти минимальный способ воспроизвести сбой
3. Отключить все warnings
4. Удалить `assert`, чтобы не мешали

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ — Викторина 1

Ответ

2. Без воспроизведения нет контролируемой отладки.

Викторина 2 — assert

Викторина

Если `assert(cond)` ложен в Debug-сборке...

1. условие тихо пропускается
2. программа аварийно останавливается (обычно)
3. компилятор исправляет код
4. возвращается `false` из `main`

Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.

Ответ — Викторина 2

Ответ

2. Assert сигнализирует нарушение допущения — abort/terminate.